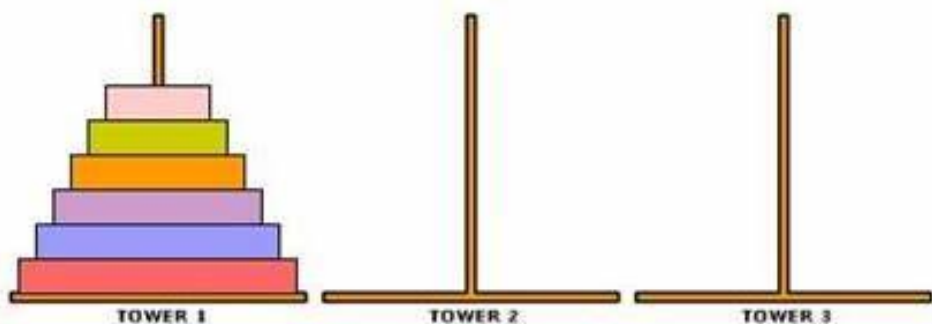


报告封面

包含实验报告名称、班级、学号、姓名、完成日期等，版面自行设计（可以整活）



报告名称：高程期中大作业



我得了玉玉症

班级：信03

学号：2351136

姓名：李盛鹏

完成日期：2024年5月14日

关于汉诺塔的综合演示

1. 1. 次级小标题（如果有，宋体，四号，加粗，段前段后间距各1行）

1. 1. 1. 三级小标题（如果有，宋体，小四，加粗，段前段后间距各1行）

1. ****基本解****: 记录每一步移动的过程，包括哪个盘子移动到哪个柱子。
2. ****基本解(步数记录)****: 在输出前加上步数信息，显示第几步进行了什么移动。
3. ****内部数组显示(横向)****: 横向打印每个柱子上的盘子情况，展示当前柱子的盘子堆叠情况。
4. ****内部数组显示(纵向+横向)****: 在横向显示的基础上，竖向打印出柱子的形状，以更形象地展示柱子和盘子的摆放情况。
5. ****图形解-预备-画三个圆柱****: 利用字符图形函数绘制出三个圆柱的形状，为后续移动做准备。
6. ****图形解-预备-在起始柱上画n个盘子****: 在柱子上利用字符图形函数画出 n 个盘子的形状，为后续移动做准备。
7. ****图形解-预备-第一次移动****: 展示环的第一次移动的图形解，让用户直观地看到盘子的移动过程。
8. ****图形解-自动移动版本****: 展示环的所有移动的图形解，让用户一次性看到整个汉诺塔问题的解决过程。
9. ****图形解-游戏版****: 制作成游戏，让用户参与汉诺塔问题的解决过程，给出不符合规则的提示，提高用户的互动体验。
0. ****退出****: 结束程序，让用户可以随时退出或结束当前的汉诺塔操作。

2. 整体设计思路

(1) 在解决汉诺塔问题时，我们首先定义了问题的基本情况：三根柱子 A、B、C，所有盘子初始都在柱子 A 上，按照大小顺序排列。我们的目标是将所有盘子从柱子 A 移至柱子 C，允许借助柱子 B 过渡。

(2) 采用经典的递归解法，对于 n 个盘子，我们递归地将 $n-1$ 个盘子从柱子 A 经由柱子 C 移至柱子 B，然后将最大的盘子从柱子 A 移至柱子 C，最后将 $n-1$ 个盘子从柱子 B 经由柱子 A 移至柱子 C，完成整个移动过程。

(3) 为了实现这一递归思路，我们设计了一个递归函数，其中参数包括盘子数量、起始柱、目标柱和中间柱。在每一步递归中，我们先移动 $n-1$ 个盘子到中间柱，然后移动最大的盘子到目标柱，最后将 $n-1$ 个盘子从中间柱移至目标柱。递归的结束条件是只有一个盘子需要移动。

(4) 为了更好地反映汉诺塔每个位置上的盘子情况，我们引入了数组的概念。通过引入全局指针 `top`，我们可以准确地反映出每个柱子上有几个盘子，从而更好地管理移动的过程。

(5) 为了让用户更直观地理解汉诺塔问题的解决过程，我们引入了图像展示。根据汉诺塔的位置关系，我们利用画了清除再画的特性，实现了图像环的移动。同时，借助数组和指针的强大功能，我们成功处理了移动位置的关系，为用户提供了更生动、直观的体验。

3. 主要功能的实现

```
//汉诺函数，负责递归移动和调用输出
void hanoi(int n, char src, char tmp, char dst, int choice)
{
    if (n == 1) {
        hanoicase(n, src, tmp, dst, choice);
        return;
    }
    else {
        hanoi(n - 1, src, dst, tmp, choice);
        hanoicase(n, src, tmp, dst, choice);
        hanoi(n - 1, tmp, src, dst, choice);
    }
}
```

最重要的汉诺函数，采取递归的思路，



1. 主函数

主函数用于启动汉诺塔问题的求解过程，设置初始参数并调用递归函数。主要任务是接收用户输入，初始化盘子数量和柱子，并开始解决汉诺塔问题。

2. 递归函数

递归函数是解决汉诺塔问题的核心。它接收盘子数量、起始柱子、目标柱子和中间柱子作为参数，并根据递归规则移动盘子。递归函数的设计要点如下：

函数参数：接收盘子数量 n 、起始柱子 src 、目标柱子 dst 和中间柱子 tmp 。

递归结束条件：当只有一个盘子时，直接将盘子从起始柱子移动到目标柱子。

递归调用：在每一步递归中，先将 $n-1$ 个盘子从起始柱子通过目标柱子移动到中间柱子，然后将最大的盘子从起始柱子移动到目标柱子，最后将 $n-1$ 个盘子从中间柱子通过起始柱子移动到目标柱子。

3. 输出函数

输出函数用于显示每次移动的过程，可以在控制台输出移动的步骤，包括从哪个柱子移动到哪个柱子。这有助于用户理解算法的执行过程。

4. 用户交互

用户交互部分可以包括向用户询问要移动的盘子数量，输出移动步骤的细节，以及在完成后询问是否继续进行下一轮移动。

4. 调试过程碰到的问题

问题 1: 数组在每次按空格重启程序的时候忘记归回初始状态, 导致一开始竖直打印数组时, 先前用的环数如果比第二次大, 在第二次的打印中会看见第一次的残骸。

解决: 在 main 最后把数组每个值都调为 0.

问题 2: 数组在每次按空格重启程序的时候忘记归回初始状态, 导致一开始竖直打印数组时, 先前用的环数如果比第二次大, 在第二次的打印中会看见第一次的残骸。

解决: 在 main 最后把数组每个值都调为 0.

问题 3: 环和柱的颜色一样。

解决: 柱的颜色为亮黄为 14, 用环的号码赋值给颜色, 就可以不和柱颜色一样

问题 4: 环的图像动的时候, 抹除的操作不正确, 导致柱子变长, 或者底座被清理

解决: 从环开始移动到最后环出现的地方, 每一步的移动实际上就是将环打印的时候, sleep 约 0.3 秒时间, 擦除环, 然后同时打印下一个位置。此时因为眼睛的延迟作用, 就会看到环连贯的移动。而为了使得柱子不变长, 就要确定柱子的打印范围, 环的下端到达柱子的顶端时, 此时做最后一次还原柱子的处理, 即 $y \geq \text{base_y} - N - 2$ 时, 画柱子。其他不画, 就可以使柱子不变长。然后底座被清理的情况则是结束位置没有选对, 在我原先坐标的 y 值-1 后, 就解决了问题。

5. 心得体会

5.1.1 基准程序要做好, 不然后面搭建的程序就很难

基准程序的重要性不言而喻。它是整个项目的基石, 决定了后续搭建程序的顺利与否。一个良好的基准程序为后续开发提供了稳固的基础, 确保了整个项目的顺利进行。通过精心设计和完善基准程序, 我们能够更好地把握项目的需求和方向, 避免后续开发中出现的种种困难和问题。基准程序的质量直接影响着整个项目的成功与否, 因此在项目初期, 务必要花费足够的时间和精力来打磨基准程序, 确保其稳定性和可靠性。只有在基准程序做好的前提下, 后续的程序搭建才能事半功倍, 为整个项目的顺利进行奠定坚实的基础。因此, 对于任何软件开发项目来说, 打造一个优秀的基准程序都是至关重要的一环。

5.1.2 在做程序的时候，要少用全局变量

全局变量在程序中使用不当带来的灾难都是致命性的，如果在程序的某处引用了和全局变量同名的变量，那么就可能改变全局变量的值，使得程序无法继续跑下去。或者是输出一大堆奇怪的结果。本人比较推崇宏定义，通过宏定义，可以对某个基准性的变量进行赋值和修改，在程序需要改变的时候，通过改变这个宏定义的值就可以轻松实现程序的维护。

5.1.3 不必要的地方不要写死

比如说 `cct_gotoxy` 的引用，带入的坐标最好是代数函数的形式，这样只要改变宏定义的数值，就可以实现对坐标整体的一个改变。这一感触还在 `for` 循环的时候有体现，`for` 的起始和终止条件，最好也不写死，而是用函数的方式表达，这样系统的维护性才高。

5.1.4 有些功能需要在开始做程序的时候就想好

比如说这次的作业中的 `choice` 来决定功能，在一开始的时候就可以用缺省函数，留好 `choice` 这个参数，这样后面的引用的修改范围就会减小很多，不容易导致错误。并且，对于 7 选项的第一次移动。写移动函数的时候就该把选项 8 全部移动给考虑到。实际上，要做好当前功能和拓展功能的一个平衡，每做一步就目光远一点想下一步，这样的程序容错性大，稳定性好。更重要的，不会因为后期修改改废了而破防。

5.2 个人偏向做好做若干小题

在程序设计中，成功的程序往往并非一蹴而就，而是需要经过前期的充分准备和铺垫，以及各功能间的良好协作。若一开始就仓促地追求完成一个庞大的程序，往往会导致思路混乱，或者在编写过程中出现断断续续、功能衔接不畅的情况。事实上，在开发程序时，应当将目标分解为一系列小任务，逐一解决，确保每个小功能都能正确运行且相互协调。重视各功能之间的联系，确保变量之间的衔接性，这是非常重要的。只有在小功能保证准确无误的基础上，才能逐步完善、扩展程序，最终形成一个整体完备的程序。

5.3 考虑小题间关系的问题

在这次的汉诺塔大作业中，每个小题之间都有着相当的关系。实话实说，我利用了这相互关系，但是由于函数功能不够独立，导致写的有点乱。下面是我对里面小题相互关系的一个总结：与汉诺解有关的函数都离不开这个递归函数，对于不同的选项，只要在 `hanoicase` 中输入 `choice` 即可做到区分。基础解（记步）建立在基础解之上，引入全局变量 `int i` 进行计数即可。横向打印盘子在 2 的基础上引入三个全局数组和表示柱子上盘子数的三个全局变量即可。纵向打印三个数组则需要计算用 `cct_gotoxy` 辅助位置

打印。

只需借助前面 4 定义好的位置，就可以完成柱子和盘子的搭建，在根据前四题数组的移动就实现了控制对应盘子的移动。

如果说想更好地利用前面的代码的话，个人认为拆分函数是一个要点。拆前面的功能成多个小函数，需要用的

时候引用，跟着逻辑写出新功能函数，做函数组装的工作即可。

```
//汉诺函数，负责递归移动和调用输出
void hanoi(int n, char src, char tmp, char dst, int choice)
{
    if (n == 1) {
        hanoicase(n, src, tmp, dst, choice);
        return;
    }
    else {
        hanoi(n - 1, src, dst, tmp, choice);
        hanoicase(n, src, tmp, dst, choice);
        hanoi(n - 1, tmp, src, dst, choice);
    }
}
```

5.4 更好地利用函数

1. 单一职责原则：每个函数应该只负责一项具体的任务，避免函数功能过于复杂。这样可以提高函数的重用性，并使代码更易于测试和调试。在汉诺塔大作业中，`hanoi` 是基准函数，用 `initial` 函数单独负责全局数组变量的初始化，在需要数组的功能里开头就引用。输入起始柱数组和结束柱数组，`My_Move` 函数专门负责数组里数值的移动。`Printshow` 则负责横着输出数组数据。`My_tower` 单独负责搭建塔的基座和竖着输出的塔。以及 `ring_move_up`、`parallel`、`down` 等单独负责了盘子的上移、平移和下移等等。

2. 参数传递：合理使用函数参数传递数据，避免过度依赖全局变量。通过参数传递数据可以使函数更具通用性，减少函数之间的耦合度。`Hanoi` 作业中，全局变量可以不用传递到函数中。但是也要注意的是函数也不要轻易地改变全局变量的值，防止最后输出错误。

3. 返回值利用：合理设计函数的返回值，确保函数能够正确地返回所需的结果。返回适当的值可以帮助其他函数更好地理解 and 利用函数的输出。比如说用一个函数解决输入的速度变量，返回一个 `Sleep` 时用的值。

4. 错误处理：在函数中实现适当的错误处理机制，确保程序在面对异常情况时能够正确处理并给出合适的反馈。写好错误处理的函数，能为后面的函数的有效数据的输入铺好路。同时将错误处理单独拎出来。能使得 `main` 函数不那么臃肿。就比方说这次汉诺塔中我用了 `input_src`, `input_dst`, `input_tmp` 来处理三个柱子的输入，其中包含了错误处理的输入。通过这三个函数我就能保证后面传递的三个柱子的参数一定是 A, B, C 而不会是别的数。还有一开始判断选择选项的函数，也通过错误处理把返回值决定在 0-9 之间。

5. 函数命名：给函数取一个清晰、具有描述性的名称，能够准确反映函数的功能。这样可以使其他开发者更容易理解代码，并快速找到需要的函数。我的函数为了和 windows 的函数不重名，一般都会加 `my_` 开头，然后用英文准确地表述名字，关于输入的，有 `input` 前缀，有关输出的，则有 `print` 前缀，移动的，有 `move` 修饰。当然，必须承认，想让函数名字简洁而有描述性确实很难，本人起名字时常很长，引用时效率不高。

6. 注释和文档：在函数定义的上方添加注释，简要描述函数的作用和功能。说明函数的输入参数、返回值以及可能的副作用。参数注释：在参数定义处添加注释，说明参数的含义、类型以及可能的取值范围。这有助于其他开发者更快地理解函数的用途。此外，编写文档，描述每个函数的功能和用法，有助于团队合作和代码维护。通过大作业，已经养成了每一小块功能函数区都加注释的习惯。

6. 附件：源程序

注意排版格式，对齐方式，可以采用两列排版，正文采用宋体，小五，行距为单倍行距

不需要全部源程序，main函数、菜单选择、输入输出错误等常规内容建议舍弃，挑部分核心代码贴上来即可。

```
//数组初始化
void initial(int n, char src, char tmp, char dst, int choice)
{
    for (int in_count = 0; in_count < n; in_count++) {
        if (src == 'A') {
            A[in_count] = n - in_count;
            B[in_count] = 0;
            C[in_count] = 0;
            topA = n;
            topB = 0;
            topC = 0;
        }
        else if (src == 'B') {
            A[in_count] = 0;
            B[in_count] = n - in_count;
            C[in_count] = 0;
            topA = 0;
            topB = n;
            topC = 0;
        }
        else {
            A[in_count] = 0;
            B[in_count] = 0;
            C[in_count] = n - in_count;
            topA = 0;
            topB = 0;
            topC = n;
        }
    }
}
```

初始化函数，前三行为按照输入环数为三个全局数组赋上环数，自数组顶端到底端分别是 1 到 n。

然后再讲全局变量附上值，让它和数组内不为零的数据也就是盘子的数量挂钩。

功能：每次开始汉诺塔前，引用这个函数来做初始化的工作。

```
//汉诺函数，负责递归移动和调用输出
void hanoi(int n, char src, char tmp, char dst, int choice)
{
    if (n == 1) {
        hanoicase(n, src, tmp, dst, choice);
        return;
    }
    else {
        hanoi(n - 1, src, dst, tmp, choice);
        hanoicase(n, src, tmp, dst, choice);
        hanoi(n - 1, tmp, src, dst, choice);
    }
}
```

最重要的汉诺函数，采用递归的思想，将问题分解为上面的 n-1 个借由结束柱搬到中间柱，然后将最底下的柱子搬到结束柱。再将 n-1 个盘子借由起始柱搬到结束柱。当只剩一个盘子的时候，将它从起始柱搬到结束柱即可。

```
//负责处理不同的模式
void hanoicase(int n, char src, char tmp, char dst, int choice)
{
    if (choice == 1) {
        i++;
        cout << setw(2) << n << "# " << src << "-->" << dst << endl;
    }
    else if (choice == 2) {
        i++;
        cout << "第" << setw(4) << i << " 步(" << setw(2) << n << "): " << src << "-->" << dst << endl;
    }
    else if (choice == 3) {
        i++;
        cout << "第" << setw(4) << i << " 步(" << setw(2) << n << "): " << src << "-->" << dst << " ";
        my_move(src, dst); //把最下面的搬到结束柱
        printshow(A, B, C, src, dst, n);
    }
    else if (choice == 4) {
        i++;
        //下面这段是暂停函数的运用
        if (i_speed == 0) { // 如果速度设置为0, 变成按回车才动一次
            my_pause();
        }
        Sleep(Selection(i_speed));
        //输出第几步是哪到哪
        cct_gotoxy(8, base_y + 6);
        cout << "第" << setw(4) << i << " 步(" << setw(2) << n << "): " << src << "-->" << dst << " ";

        my_move(src, dst); //把最下面的搬到结束柱
        my_tower(choice);
        cct_gotoxy(8 + 21, base_y + 6);
        printshow(A, B, C, src, dst, n);
    }
    else if (choice == 7) {
        i++;
        if (i == 1) {
            ring_move(n, src, dst, choice);
            my_move(src, dst);
        }
    }
    else if (choice == 8) {
        i++;
        if (i_speed == 0) { // 如果速度设置为0, 变成按回车才动一次
            my_pause();
        }
        Sleep(Selection(i_speed));
        ring_move(n, src, dst, choice);
        my_move(src, dst); //把最下面的搬到结束柱
        cct_gotoxy(0, base_y + N + 7);
        cout << "第" << setw(4) << i << " 步(" << setw(2) << n << "): " << src << "-->" << dst << " ";

        my_tower(choice);
        cct_gotoxy(0 + 21, base_y + N + 7);
        printshow(A, B, C, src, dst, n);
        cct_gotoxy(60, base_y + N + 9);
    }
}
}
```

这是我的 hanoicase 函数, 根据不同的 choice 输入给出了不同的处理方案, 但是每一个 choice 之间又有部分重叠部分, 是还可以接着优化的。

```
//用来做柱子图像的函数
void tower_setup()
{
    int y=base_y;
    //底座的搭建
    for (int count = 0; count < 3; count++) {
        cct_showch(1+count*(2*(N+1)+1+9), base_y, ' ', COLOR_HYELLOW, COLOR_HYELLOW, 2 * (N + 1) + 1);
    }

    //柱子的搭建
    for (int y = base_y; y > base_y-N-2; y--) {
        for (int count = 0; count < 3; count++) {
            cct_showch(1 + count * (2 * (N+1) + 1 + 9) + N+1, y-1, ' ', COLOR_HYELLOW, COLOR_HYELLOW, 1);
            Sleep(100);
        }
    }

    //恢复缺省颜色
    cct_setcolor();
    cct_gotoxy(0, base_y+base_y/4);
}
```

这个函数 `tower_setup()` 的目的是绘制一个柱状图像，包括底座和柱子。函数首先在底座位置绘制三个黄色空格，每个宽度为 $2 * (N + 1) + 1$ ，这个循环会在底座位置循环执行三次。接着，在底座上方逐渐向上绘制三根柱子，每根柱子的宽度为 1，颜色也是亮黄色。这个过程通过两个嵌套的循环实现，外层循环控制高度，内层循环控制每根柱子的绘制。在绘制完成后，函数会恢复默认颜色设置，并将光标移动到一个特定的位置 $(0, base_y + base_y/4)$ 。

```
//用来做盘子的函数
void ring_setup(int n, char src)
{
    int tamp = n; //为了让变量n不进入循环，用这个tamp来做中间变量
    for (int y = base_y; y > base_y - tamp; y--) {
        cct_showch(1+(N+1-n) + (2 * (N+1) + 1 + 9) * (src - 'A'), y - 1, ' ', n, n, 2 * n + 1);
        n--;
        Sleep(100);
    }
    cct_setcolor();
    cct_gotoxy(0, base_y + base_y / 4);
}
```

这段代码是 `ring_setup` 的函数，接受两个参数 `n` 和 `src`。函数的作用是根据传入的参数在屏幕上显示一系列盘子。首先，将参数 `n` 赋值给变量 `tamp`，然后通过一个循环逐行显示空格，每行的空格数量递减，直到最后一行。在每一行显示空格的过程中，会调用 `cct_showch` 函数来显示盘子，并通过 `Sleep(100)` 函数让程序暂停 100 毫秒，以便观察每一行的变化。循环结束后，会调用 `cct_setcolor` 函数来恢复默认颜色设置，并将光标移动到特定位置 $(0, base_y + base_y / 4)$ 。

```
//上移函数
void ring_move_up(int n, char src, int choice)
{
    int y;
    if (src == 'A') {
        y = base_y - topA;
    }
    else if (src == 'B') {
        y = base_y - topB;
    }
    else {
        y = base_y - topC;
    }

    /* 将一串字符从下向上移动 */
    //画到min_y, 比min_y大的都要在画完后停0.3秒清除, 柱子的长是N+2, 所以比base_y-N-2-1大的都要清了重画
    for (y; y > min_y-1; y--) {
        cct_showstr(1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (src - 'A'), y, "", n, n, 2*n+1);

        if (choice == 8) {
            Sleep(Selection(i_speed));
        }
        else { /* 延时0.3秒 */
            Sleep(300);
        }
    }

    if (y > min_y) {
        /* 清除显示(最后一次保留), 清除方法为用正常颜色+空格重画一遍刚才的位置 */
        cct_showch(1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (src - 'A'), y, ' ', COLOR_BLACK, COLOR_WHITE, 2*n+1);
    }
    if (y > base_y - N - 2 - 1) {
        cct_showch(1 + (N + 1) + (2 * (N + 1) + 1 + 9) * (src - 'A'), y, ' ', COLOR_HYELLOW, COLOR_HYELLOW, 1);
    }
    //恢复缺省颜色
    cct_setcolor();
    cct_gotoxy(0, base_y + base_y / 4);
} //end of for
```

这个是盘子的上移函数

```
//平移函数
void ring_move_parallel(int n, char src, char dst, int choice)
{
    int x;
    if (src == 'A' || (src == 'B' && dst == 'C')) {
        for (x = 1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (src - 'A'); x <= 1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (dst - 'A'); x++) {
            cct_showstr(x, min_y, " ", n, n, 2 * n + 1);

            if (choice == 8) {
                Sleep(Selection(i_speed));
            }
            else {
                /* 延时0.1秒 */
                Sleep(100);
            }

            /* 在产生的同时将比dst所在柱x坐标小的都清除 */
            if (x < 1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (dst - 'A')) {
                cct_showch(x, min_y, ' ', COLOR_BLACK, COLOR_WHITE, 2 * n + 1);
            }

            //恢复缺省颜色
            cct_setcolor(0);
            cct_gotoxy(0, base_y + base_y / 4);
        }

    }
    else if (src == 'C' || (src == 'B' && dst == 'A')) {
        for (x = 1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (src - 'A'); x >= 1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (dst - 'A'); x--) {
            cct_showstr(x, min_y, " ", n, n, 2 * n + 1);

            if (choice == 8) {
                Sleep(Selection(i_speed));
            }
            else {
                /* 延时0.1秒 */
                Sleep(100);
            }

            /* 在产生的同时将比dst所在柱x坐标大的都清除 */
            if (x > 1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (dst - 'A')) {
                cct_showstr(x, min_y, " ", COLOR_BLACK, COLOR_WHITE, 2 * n + 1);
            }

            //恢复缺省颜色
            cct_setcolor(0);
            cct_gotoxy(0, base_y + base_y / 4);
        }
    }
}
```

这个是平移函数

```
//下移函数
void ring_move_down(int n, char dst, int choice)
{
    //用一个y来负责动态变化的y坐标值，用tamp来做y的终点
    int y, tamp=0;
    if (dst == 'A') {
        tamp = base_y - topA - 1;
    }
    else if (dst == 'B') {
        tamp = base_y - topB - 1;
    }
    else if (dst == 'C') {
        tamp = base_y - topC - 1;
    }

    //下落过程，是从min_y到(base_y-topx-1)的过程中，先打印环，再等300ms消去环，只留下最后位置的环
    for (y = min_y; y <= tamp; y++) {
        cct_showstr(1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (dst - 'A'), y, " ", n, n, 2 * n + 1);

        if (choice == 8) {
            Sleep(Selection(i_speed));
        }
        else {
            /* 延时0.3秒 */
            Sleep(300);
        }
    }
}
```

```

    if (y < tamp) {
        cct_showch(1 + (N + 1 - n) + (2 * (N + 1) + 1 + 9) * (dst - 'A'), y, ' ', COLOR_BLACK, COLOR_WHITE, 2 * n + 1);
    }
    if (y >= base_y - N - 2 && y < tamp) {
        cct_showch(1 + (N + 1) + (2 * (N + 1) + 1 + 9) * (dst - 'A'), y, ' ', COLOR_HYELLOW, COLOR_HYELLOW, 1);
    }
    //恢复缺省颜色
    cct_setcolor();
    cct_gotoxy(0, base_y + base_y / 4);
}

```

这是下移函数（上半部分在上页），上移，平移，下移函数共同组成了我的盘子移动联合函数 ring_move。

```

bool ring_can_move(char src, char dst)
{
    if (src == 'A' && dst == 'C') {
        if (A[topA - 1] > C[topC - 1] && C[topC - 1] > 0) {
            return false;
        }
        else {
            return true;
        }
    }
    else if (src == 'A' && dst == 'B') {
        if (A[topA - 1] > B[topB - 1] && B[topB - 1] > 0) {
            return false;
        }
        else {
            return true;
        }
    }
    else if (src == 'B' && dst == 'C') {
        if (B[topB - 1] > C[topC - 1] && C[topC - 1] > 0) {
            return false;
        }
        else {
            return true;
        }
    }
}

```

```

    else if (src == 'B' && dst == 'A') {
        if (B[topB - 1] > A[topA - 1] && A[topA - 1] > 0) {
            return false;
        }
        else {
            return true;
        }
    }
    else if (src == 'C' && dst == 'B') {
        if (C[topC - 1] > B[topB - 1] && B[topB - 1] > 0) {
            return false;
        }
        else {
            return true;
        }
    }
    else if (src == 'C' && dst == 'A') {
        if (C[topC - 1] > A[topA - 1] && A[topA - 1] > 0) {
            return false;
        }
        else {
            return true;
        }
    }
    return true;
}

```

```

bool ring_exist(char src)
{
    if (src == 'A') {
        if (topA == 0) {
            return false;
        }
        else {
            return true;
        }
    }
    if (src == 'B') {
        if (topB == 0) {
            return false;
        }
        else {
            return true;
        }
    }
    if (src == 'C') {
        if (topC == 0) {
            return false;
        }
        else {
            return true;
        }
    }
    return true;
}

```

这个是我的游戏中的是否可以移动的程序 ring_can_move 和盘子是否存在程序 ring_exist, 这两个函数的返回值能决定游戏中是否报操作错误提示。

```
void hanoi_game(int choice, char game_dst, int n)
{
    char src, tmp, dst;
    bool not_exit = true;

    while (not_exit) {
        //堆满了, 游戏结束
        if (game_dst == 'A') {
            if (topA == n) {
                cct_gotoxy(0, base_y + N + 10);
                cout << "游戏中止!!!!!" << endl;
                break;
            }
        }
        if (game_dst == 'B') {
            if (topB == n) {
                cct_gotoxy(0, base_y + N + 10);
                cout << "游戏中止!!!!!" << endl;
                break;
            }
        }
        if (game_dst == 'C') {
            if (topC == n) {
                cct_gotoxy(0, base_y + N + 10);
                cout << "游戏中止!!!!!" << endl;
                break;
            }
        }
    }
}
```

```
src = getchar();
//结束条件
if (src == 'Q') {
    cct_gotoxy(0, base_y + N + 10);
    cout << "游戏中止!!!!!" << endl;
    not_exit = false;
}

dst = getchar();
if (src == 'a' || src == 'b' || src == 'c') {
    src -= 32;
}
if (dst == 'a' || dst == 'b' || dst == 'c') {
    dst -= 32;
}
if ((src == 'A' && (dst == 'B' || dst == 'C')) || (src == 'B' && (dst == 'A' || dst == 'C')) || (src == 'C' && (dst == 'B' || dst == 'A'))) {
    tmp = input_tmp(src, dst);
}

cin.ignore(65536, '\n');
//清除输入
cct_gotoxy(60, base_y + N + 9);
cout << " ";
cct_gotoxy(60, base_y + N + 9);

if (ring_exist(src) && ring_can_move(src, dst)) {
    ring_move(0, src, dst, choice);
    i++;
    cct_gotoxy(0, base_y + N + 7);
}
```

```

        cout << "第" << setw(4) << i << " 步(" << setw(2) << n << "):" << src << "→" << dst << " ";

        my_move(src, dst); //把最下面的搬到结束柱
        my_tower(choice);
        cct_gotoxy(0 + 21, base_y + N + 7);
        printshow(A, B, C, src, dst, n);
        cct_gotoxy(60, base_y + N + 9);
    }

    else if (!ring_exist(src)) {
        cct_gotoxy(0, base_y + N + 10);
        cout << "柱源为空! ";
        Sleep(1000);
        cct_gotoxy(0, base_y + N + 10);
        cout << " ";
        cct_gotoxy(60, base_y + N + 9);
    }

    else {
        cct_gotoxy(0, base_y + N + 10);
        cout << "大盘压小盘, 非法移动! ";
        Sleep(1000);
        cct_gotoxy(0, base_y + N + 10);
        cout << " ";
        cct_gotoxy(60, base_y + N + 9);
    }
}

return;

```

这个是我最后的游戏的函数。

总结

这次写实验报告大作业，一开始五一那阵子都在疯玩，后面写的时间也很仓促，会有写的不好的地方。通过一个相对大程序的完成，我有了不同往日的感想，我意识到编程需要高度的习惯性和规范性。任何不良习惯都可能在大型程序中造成严重问题。混杂的函数功能、缺乏条理、忘记部分代码、缺乏注释，重复无用代码多等问题都需要平时的习惯养成来克服。

学代码要讲究一个融会贯通，前面的递归知识和后面的数组等的知识的结合，反应了我个人的学习思维方式的提升。不仅如此，如何在相似的项目中调整逻辑顺序，又对我的思维转化来了一波不小的挑战。今后，期待着自己坐下来，静得下来，理得顺思路，写更优质的码。